

Laborator 4 — Aplicații ale rețelei neuronale recurente (RNN)

Corețchi Mihai

Set de date: perechi de propoziții RO-EN (Tatoeba / Anki — manythings.org/anki/ron-eng.zip)

Obiectiv: identificarea limbii (**română / engleză**) a unei propoziții folosind o rețea neuronală recurentă (RNN), cu comparație față de **LSTM** și **GRU** (variantele uni- și **Bidirectional**).

Instalarea dependențelor

```
In [1]: %pip install -q tensorflow pandas numpy matplotlib seaborn scikit-learn
```

1. Importul bibliotecilor și configurarea mediului

```
In [2]: import os
import io
import time
import zipfile
import urllib.request

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import (
    Input, TextVectorization, Embedding,
    RNN, SimpleRNNCell, LSTM, GRU, Bidirectional, Dense,
)
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report, f1_score,
)

%matplotlib inline
sns.set_theme(style="whitegrid")
tf.keras.utils.set_random_seed(101)

print("TensorFlow:", tf.__version__)
```

2. Obținerea și analiza exploratorie a datelor

Folosim corpusul **Tatoeba / Anki** de perechi de propoziții engleză-română (`ron-eng.zip` de pe `manythings.org`). Fiecare linie din `ron.txt` are forma `propoziție_EN \t propoziție_R0 \t atribuire CC-BY`.

Din fiecare pereche construim **două exemple**: propoziția engleză (etichetă `0`) și cea românească (etichetă `1`). Rezultă un set **perfect echilibrat** (același număr de propoziții pe limbă), potrivit pentru o clasificare binară cu ieșire `sigmoid`.

```
In [3]: DATA_DIR = "data"
os.makedirs(DATA_DIR, exist_ok=True)
ron_txt = os.path.join(DATA_DIR, "ron.txt")

# manythings.org respinge cererile fără antet de browser (HTTP 406/403),
# de aceea trimitem un set complet de antete (User-Agent, Accept, Referer).
if not os.path.exists(ron_txt):
    url = "http://www.manythings.org/anki/ron-eng.zip"
    headers = {
        "User-Agent": ("Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                       "AppleWebKit/537.36 (KHTML, like Gecko) "
                       "Chrome/120.0 Safari/537.36"),
        "Accept": ("text/html,application/xhtml+xml,application/xml;"
                   "q=0.9,image/avif,image/webp,*/*;q=0.8"),
        "Accept-Language": "en-US,en;q=0.9",
        "Referer": "https://www.manythings.org/anki/",
    }
    req = urllib.request.Request(url, headers=headers)
    blob = urllib.request.urlopen(req, timeout=120).read()
    with zipfile.ZipFile(io.BytesIO(blob)) as z:
        z.extract("ron.txt", DATA_DIR)
    print("Descărcat și extras:", ron_txt)
else:
    print("Fișier deja prezent:", ron_txt)

# Dacă descărcarea automată este blocată de rețea: descărcați manual
# http://www.manythings.org/anki/ron-eng.zip și puneți `ron.txt` în folderul
```

Descărcat și extras: data/ron.txt

```
In [4]: # Parsare: col. 0 = EN (eticheta 0), col. 1 = RO (eticheta 1)
LANGS = ["Engleză", "Română"]
texts, labels = [], []
with open(ron_txt, encoding="utf-8") as f:
    for line in f:
        parts = line.rstrip("\n").split("\t")
        if len(parts) < 2:
            continue
        en, ro = parts[0].strip(), parts[1].strip()
        if en and ro:
            texts.append(en); labels.append(0)
```

```

        texts.append(ro); labels.append(1)

df = pd.DataFrame({"text": texts, "label": labels})
df["lang"] = df["label"].map(lambda i: LANGS[i])
df["n_words"] = df["text"].str.split().str.len()

print("Total propoziții:", len(df))
print(df["lang"].value_counts())
df.sample(5, random_state=101)

```

```

Total propoziții: 32594
lang
Engleză      16297
Română       16297
Name: count, dtype: int64

```

Out [4]:

	text	label	lang	n_words
3866	Don't deceive him.	0	Engleză	3
4815	Nu râde de el.	1	Română	4
6247	A plouat săptămâna trecută.	1	Română	4
2692	They ignored me.	0	Engleză	3
23194	Tom would've liked to stay longer.	0	Engleză	6

In [5]:

```

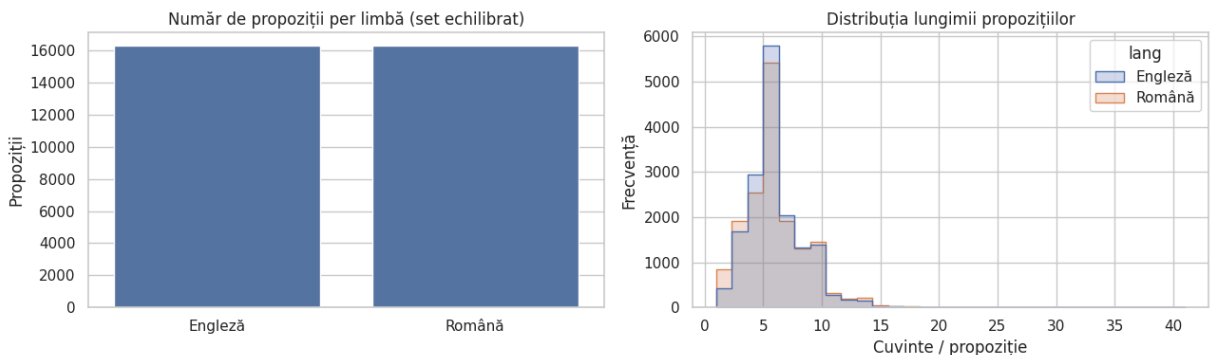
fig, axes = plt.subplots(1, 2, figsize=(13, 4))

sns.countplot(data=df, x="lang", ax=axes[0])
axes[0].set_title("Număr de propoziții per limbă (set echilibrat)")
axes[0].set_xlabel(""); axes[0].set_ylabel("Propoziții")

sns.histplot(data=df, x="n_words", hue="lang", bins=30,
              element="step", ax=axes[1])
axes[1].set_title("Distribuția lungimii propozițiilor")
axes[1].set_xlabel("Cuvinte / propoziție"); axes[1].set_ylabel("Frecvență")

plt.tight_layout()
plt.show()

```



In [6]:

```

for lang in LANGS:
    print(f"\n- Exemple «{lang}» -")

```

```
for s in df.loc[df["lang"] == lang, "text"].sample(5, random_state=7):
    print("  •", s)
```

- Exemple «Engleză» –
 - Did you break the lamp?
 - She took full advantage of her stay in London to improve her English.
 - We now know that the testimony he gave was coerced.
 - This is where it's all happening.
 - Take care!
- Exemple «Română» –
 - Ai stricat lampa?
 - A profitat din plin de șederea ei în Londra pentru a-și îmbunătăți engleza.
 - Acum știm că mărturia pe care a dat-o a fost una forțată.
 - Aici este locul în care se petrece totul.
 - Ai grijă de tine!

3. Pregătirea datelor

- **Split stratificat 70/15/15** cu `sklearn.model_selection.train_test_split` (aplicat de două ori, `random_state=101`, `stratify=y`) → fiecare subset păstrează proporția egală română/engleză.
- Construim un pipeline `tf.data` cu `prefetch(AUTOTUNE)` (suprapune citirea datelor cu calculul), exact ca la laboratoarele anterioare. Textul intră ca șir brut de formă `(batch, 1)` — vectorizarea se face **în interiorul modelului**.
- `TextVectorization` transformă textul în secvențe de indici întregi (`output_mode="int"`), trunchiate/completate la `MAX_LEN`. **Adaptăm vocabularul doar pe textele de antrenare** (română + engleză împreună) → rezultă un **vocabular comun bilingv**. Standardizarea implicită păstrează diacriticele românești (ă â î ș ț), semnal puternic pentru limbă.

```
In [7]: # Split stratificat 70/15/15 cu scikit-learn (train_test_split aplicat de 2
X = df["text"].to_numpy()
y = df["label"].to_numpy()

X_train, X_tmp, y_train, y_tmp = train_test_split(
    X, y, test_size=0.30, random_state=101, stratify=y,
)
X_val, X_test, y_val, y_test = train_test_split(
    X_tmp, y_tmp, test_size=0.50, random_state=101, stratify=y_tmp,
)

splits = {
    "train": (X_train, y_train),
    "val": (X_val, y_val),
    "test": (X_test, y_test),
}
for name, (xs, ys) in splits.items():
    print(f"{name:5s}: {len(xs):6d} propoziții "
          f"(EN={int((ys == 0).sum())}, RO={int((ys == 1).sum())})")
```

```

BATCH_SIZE = 64
AUTOTUNE = tf.data.AUTOTUNE

def make_ds(xs, ys, shuffle=False):
    # forma (N, 1) – potrivită pentru Input(shape=(1,), dtype="string")
    ds = tf.data.Dataset.from_tensor_slices((xs.reshape(-1, 1), ys))
    if shuffle:
        ds = ds.shuffle(len(xs), seed=101)
    return ds.batch(BATCH_SIZE).prefetch(AUTOTUNE)

train_ds = make_ds(*splits["train"], shuffle=True)
val_ds = make_ds(*splits["val"])
test_ds = make_ds(*splits["test"])

```

```

train: 22815 propoziții (EN=11408, RO=11407)
val : 4889 propoziții (EN=2444, RO=2445)
test : 4890 propoziții (EN=2445, RO=2445)

```

In [8]:

```

MAX_TOKENS = 20000
MAX_LEN = 50

vectorize_layer = TextVectorization(
    max_tokens=MAX_TOKENS,
    output_mode="int",
    output_sequence_length=MAX_LEN,
)
# Adaptăm DOAR pe textele de antrenare (RO + EN) → vocabular comun bilingv.
vectorize_layer.adapt(splits["train"][0])

vocab = vectorize_layer.get_vocabulary()
print("Mărime vocabular:", len(vocab))
print("Primii 30 tokeni:", vocab[2:32])

```

```

Mărime vocabular: 12830
Primii 30 tokeni: [np.str_('tom'), np.str_('a'), np.str_('i'), np.str_('th
e'), np.str_('to'), np.str_('să'), np.str_('nu'), np.str_('de'), np.str_('yo
u'), np.str_('is'), np.str_('este'), np.str_('că'), np.str_('în'), np.str_
('that'), np.str_('am'), np.str_('o'), np.str_('la'), np.str_('mary'), np.st
r_('he'), np.str_('in'), np.str_('was'), np.str_('of'), np.str_('asta'), np.
str_('pe'), np.str_('cu'), np.str_('do'), np.str_('are'), np.str_('it'), np.
str_('ce'), np.str_('un')]

```

In [9]:

```

# Dovada vocabularului bilingv: tokeni clar românești ȘI clar englezești
ro_tokens = [w for w in vocab if any(c in w for c in "ăâîșțĂÂÎȘȚ")][:15]
en_tokens = [w for w in ["the", "and", "you", "what", "where", "is", "not",
                        if w in vocab]
print("Tokeni clar românești (cu diacritice):", ro_tokens)
print("Tokeni clar englezești:", en_tokens)
print("→ Vocabularul este comun pentru ambele limbi (română + engleză).")

```

Tokeni clar românești (cu diacritice): [np.str_('să'), np.str_('că'), np.str_('în'), np.str_('mă'), np.str_('și'), np.str_('făcut'), np.str_('când'), np.str_('ești'), np.str_('așa'), np.str_('îmi'), np.str_('poți'), np.str_('dacă'), np.str_('această'), np.str_('facă'), np.str_('știe')]

Tokeni clar englezești: ['the', 'and', 'you', 'what', 'where', 'is', 'not', 'have']

→ Vocabularul este comun pentru ambele limbi (română + engleză).

4. Crearea modelului RNN de bază (SimpleRNN)

Construim un model `Sequential`:

- **`Input(shape=(1,), dtype="string")`** — modelul primește text brut; toată preprocesarea este integrată, deci predicția în timp real se face direct pe propoziții.
- **`TextVectorization`** (adaptat la pasul 3) → indici întregi.
- **`Embedding(MAX_TOKENS+1, 128)`** — fiecare token devine un vector dens de 128 dimensiuni, învățat odată cu rețeaua (cuvinte cu rol similar ajung apropiate în spațiu).
- **`RNN(SimpleRNNCell(64))`** — celula recurentă parcurge secvența și menține o **stare ascunsă** de 64 de unități care „rezumă” propoziția citită până la momentul curent.
- **`Dense(64, relu)`** apoi **`Dense(1, sigmoid)`** — capul de clasificare; ieșire o singură probabilitate ($\approx 1 \rightarrow$ română, $\approx 0 \rightarrow$ engleză).

Funcția `build_model` reutilizează **același** `vectorize_layer` și aceeași structură, schimbând doar stratul recurent — astfel comparația dintre SimpleRNN / LSTM / GRU este corectă (singura variabilă este tipul de recurență).

```
In [10]: def build_model(recurrent_layer, name):
# Model identic; variază doar stratul recurent. Reutilizează
# vectorize_layer adaptat (vocabular bilingv comun).
return Sequential([
    Input(shape=(1,), dtype="string"),
    vectorize_layer,
    Embedding(MAX_TOKENS + 1, 128, name="embedding"),
    recurrent_layer,
    Dense(64, activation="relu"),
    Dense(1, activation="sigmoid"),
], name=name)

rnn_model = build_model(
    RNN(SimpleRNNCell(64), return_sequences=False, return_state=False),
    name="SimpleRNN",
)
rnn_model.summary()
```

Model: "SimpleRNN"

Layer (type)	Output Shape	Par
text_vectorization (TextVectorization)	(None, 50)	
embedding (Embedding)	(None, 50, 128)	2,560
rnn (RNN)	(None, 64)	12
dense (Dense)	(None, 64)	4
dense_1 (Dense)	(None, 1)	

Total params: 2,576,705 (9.83 MB)

Trainable params: 2,576,705 (9.83 MB)

Non-trainable params: 0 (0.00 B)

5. Compilarea și antrenarea modelului de bază

- **Optimizer:** Adam(1e-3) — robust, combină momentum și rate adaptive.
- **Loss:** binary_crossentropy — clasificare binară (engleză vs română) cu ieșire sigmoid.
- **Metrică:** accuracy.
- **EarlyStopping** pe val_accuracy cu restore_best_weights=True — oprește antrenarea când validarea nu mai progresează și păstrează cele mai bune greutăți (alege automat numărul optim de epoci).

Helper-ele compile_fit și plot_history vor fi refolosite pentru toate modelele comparate.

```
In [11]: def compile_fit(model, epochs=10):
    model.compile(
        optimizer=Adam(learning_rate=1e-3),
        loss="binary_crossentropy",
        metrics=["accuracy"],
    )
    early_stop = EarlyStopping(
        monitor="val_accuracy", patience=3,
        restore_best_weights=True, mode="max",
    )
    t0 = time.perf_counter()
    history = model.fit(
        train_ds, validation_data=val_ds,
        epochs=epochs, callbacks=[early_stop], verbose=2,
    )
    return history, time.perf_counter() - t0

def plot_history(history, title):
    hist = pd.DataFrame(history.history)
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

```

axes[0].plot(hist["accuracy"], label="train accuracy")
axes[0].plot(hist["val_accuracy"], label="val accuracy")
axes[0].set_title(f"{title} - accuracy")
axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Accuracy"); axes[0].leg

axes[1].plot(hist["loss"], label="train loss")
axes[1].plot(hist["val_loss"], label="val loss")
axes[1].set_title(f"{title} - loss")
axes[1].set_xlabel("Epoch"); axes[1].set_ylabel("Loss"); axes[1].legend(

plt.tight_layout()
plt.show()

rnn_history, rnn_time = compile_fit(rnn_model)
print(f"Timp antrenare SimpleRNN: {rnn_time:.1f} s")
plot_history(rnn_history, "SimpleRNN")

```

Epoch 1/10

357/357 - 34s - 95ms/step - accuracy: 0.9767 - loss: 0.0644 - val_accuracy: 0.9959 - val_loss: 0.0139

Epoch 2/10

357/357 - 15s - 41ms/step - accuracy: 0.9992 - loss: 0.0043 - val_accuracy: 0.9980 - val_loss: 0.0080

Epoch 3/10

357/357 - 14s - 41ms/step - accuracy: 0.9997 - loss: 0.0013 - val_accuracy: 0.9969 - val_loss: 0.0155

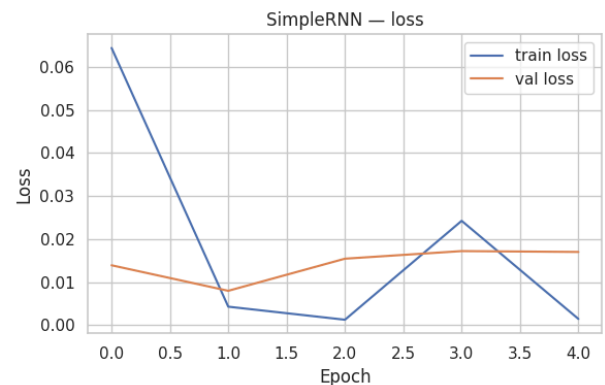
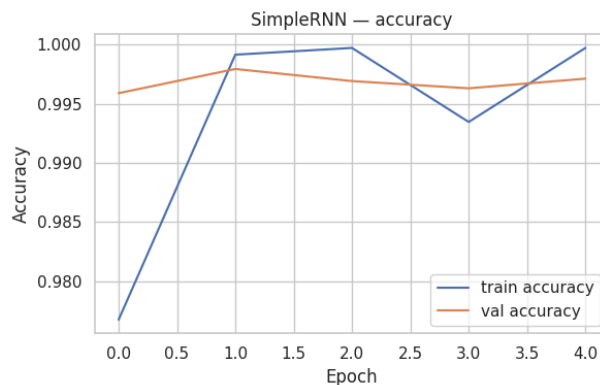
Epoch 4/10

357/357 - 14s - 40ms/step - accuracy: 0.9935 - loss: 0.0243 - val_accuracy: 0.9963 - val_loss: 0.0172

Epoch 5/10

357/357 - 15s - 41ms/step - accuracy: 0.9997 - loss: 0.0015 - val_accuracy: 0.9971 - val_loss: 0.0170

Timp antrenare SimpleRNN: 91.9 s



6. Evaluarea modelului de bază

Evaluăm pe setul de **test** (nevăzut la antrenare): loss, acuratețe, F1 (macro/weighted), raport pe clase, matricea de confuzie și câteva predicții concrete pe propoziții din test.

```

In [12]: def evaluate_model(model):
          y_true = np.concatenate([yb.numpy() for _, yb in test_ds])
          y_prob = model.predict(test_ds, verbose=0).ravel()

```

```

y_pred = (y_prob >= 0.5).astype(int)
return y_true, y_pred, y_prob

y_true, y_pred, _ = evaluate_model(rnn_model)
test_loss, test_acc = rnn_model.evaluate(test_ds, verbose=0)

print(f"Test loss:      {test_loss:.4f}")
print(f"Test accuracy: {test_acc:.4f}")
print(f"F1-macro:      {f1_score(y_true, y_pred, average='macro'):.4f}")
print(f"F1-weighted:    {f1_score(y_true, y_pred, average='weighted'):.4f}")
print()
print(classification_report(y_true, y_pred, target_names=LANGS, zero_division=0))

```

```

Test loss:      0.0030
Test accuracy:  0.9992
F1-macro:      0.9992
F1-weighted:    0.9992

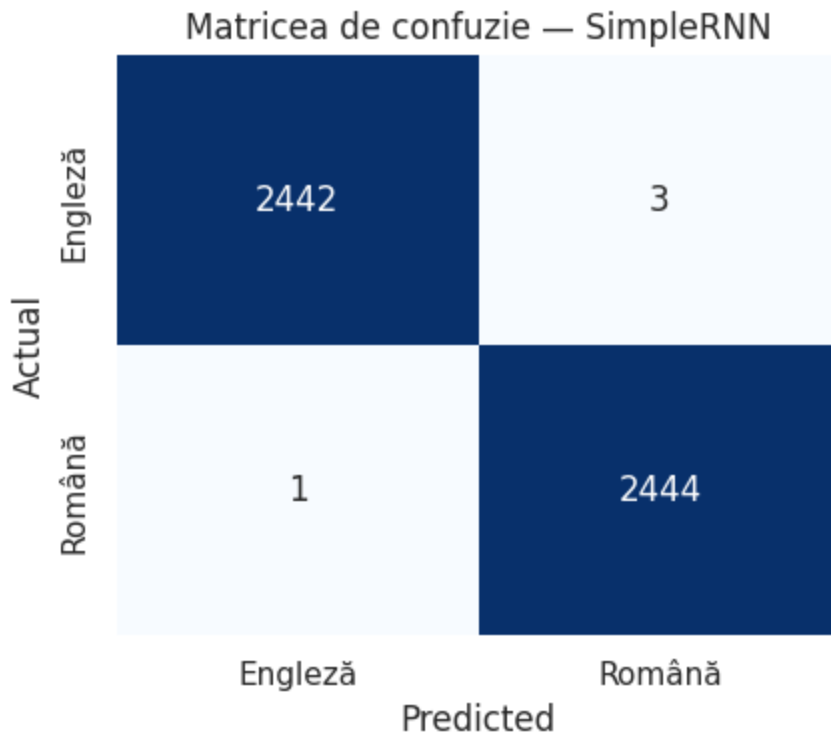
```

	precision	recall	f1-score	support
Engleză	1.00	1.00	1.00	2445
Română	1.00	1.00	1.00	2445
accuracy			1.00	4890
macro avg	1.00	1.00	1.00	4890
weighted avg	1.00	1.00	1.00	4890

```

In [13]: cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(4.5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=LANGS, yticklabels=LANGS, cbar=False)
plt.title("Matricea de confuzie - SimpleRNN")
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.tight_layout()
plt.show()

```



```
In [14]: # Keras 3 nu acceptă direct array-uri NumPy de string-uri (object dtype);
# le convertim într-un tensor tf.string de formă (N, 1) – exact ce produce
# și pipeline-ul tf.data. Helper-ul e refolosit la predicția în timp real.
def to_input(texts):
    return tf.constant(np.asarray(texts, dtype=str).reshape(-1, 1))

# Predicții concrete pe 10 propoziții aleatorii din setul de test
sx, sy = splits["test"]
pick = np.random.default_rng(1).choice(len(sx), 10, replace=False)
preds = (rnn_model.predict(to_input(sx[pick]), verbose=0).ravel() >= 0.5).as

for i, p in zip(pick, preds):
    mark = "✓" if sy[i] == p else "✗"
    print(f"[{mark}] real={LANGS[sy[i]]:8s} pred={LANGS[p]:8s} | {sx[i][:70]}")
```

```
[✓] real=Engleză pred=Engleză | Let's play a game of baseball.
[✓] real=Română pred=Română | Am o răceală acum.
[✓] real=Engleză pred=Engleză | Do you want to be left out?
[✓] real=Română pred=Română | Lasă-l pe Tom să intre.
[✓] real=Română pred=Română | Pune-ți jucăriile la loc.
[✓] real=Engleză pred=Engleză | It's important to take care of your skin.
[✓] real=Engleză pred=Engleză | Tom and Mary know John didn't do that.
[✓] real=Română pred=Română | Te rog să-ți aduci aminte să pui această s
crisoare la poștă în drum sp
[✓] real=Română pred=Română | Ai văzut ce a făcut Tom?
[✓] real=Engleză pred=Engleză | Tom is half asleep.
```

7. Comparăție cu alte modele RNN

Antrenăm, în aceleași condiții (aceleași vocabular, Embedding, cap dens, optimizator și `EarlyStopping`), patru variante recurente alternative. `SimpleRNN` suferă de

dispariția gradientului pe secvențe lungi; **LSTM** și **GRU** folosesc **porți** care rețin/uită informație selectiv. Varianta **Bidirectional** citește propoziția în ambele sensuri (stânga→dreapta și dreapta→stânga), utilă fiindcă indicii de limbă pot apărea oriunde în propoziție.

7.1 LSTM

```
In [15]: lstm_model = build_model(LSTM(64), name="LSTM")
lstm_history, lstm_time = compile_fit(lstm_model)
print(f"Timp antrenare LSTM: {lstm_time:.1f} s")
plot_history(lstm_history, "LSTM")
```

Epoch 1/10

357/357 – 32s – 89ms/step – accuracy: 0.4962 – loss: 0.6935 – val_accuracy: 0.4999 – val_loss: 0.6932

Epoch 2/10

357/357 – 28s – 77ms/step – accuracy: 0.4950 – loss: 0.6933 – val_accuracy: 0.5001 – val_loss: 0.6934

Epoch 3/10

357/357 – 27s – 77ms/step – accuracy: 0.5031 – loss: 0.6933 – val_accuracy: 0.4999 – val_loss: 0.6934

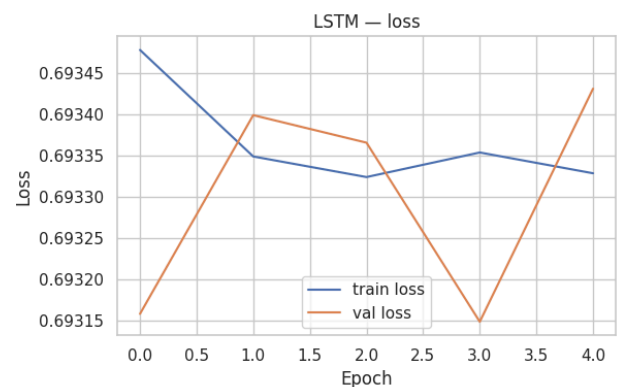
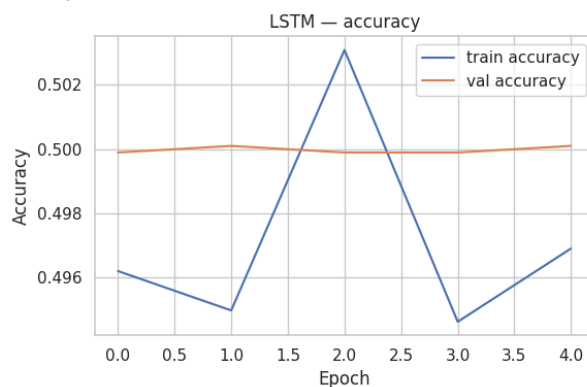
Epoch 4/10

357/357 – 27s – 76ms/step – accuracy: 0.4946 – loss: 0.6934 – val_accuracy: 0.4999 – val_loss: 0.6931

Epoch 5/10

357/357 – 42s – 119ms/step – accuracy: 0.4969 – loss: 0.6933 – val_accuracy: 0.5001 – val_loss: 0.6934

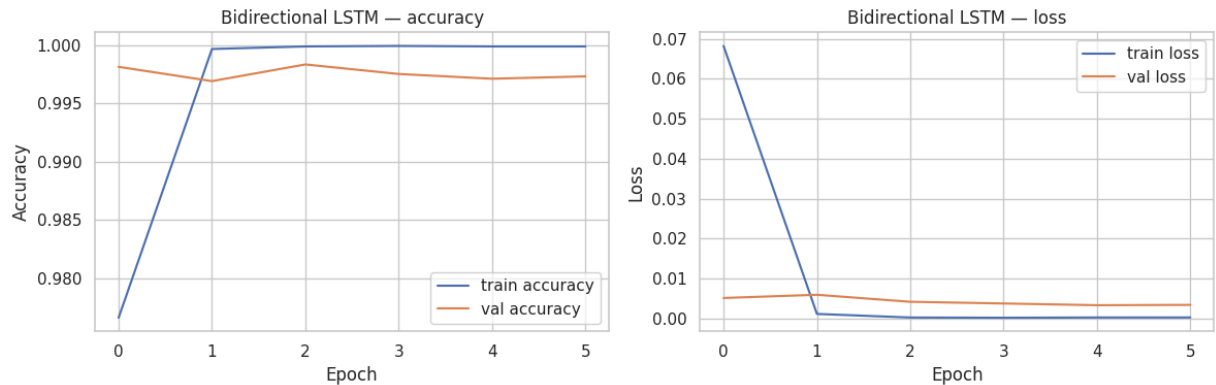
Timp antrenare LSTM: 156.5 s



7.2 Bidirectional LSTM

```
In [16]: bilstm_model = build_model(Bidirectional(LSTM(64)), name="BiLSTM")
bilstm_history, bilstm_time = compile_fit(bilstm_model)
print(f"Timp antrenare BiLSTM: {bilstm_time:.1f} s")
plot_history(bilstm_history, "Bidirectional LSTM")
```

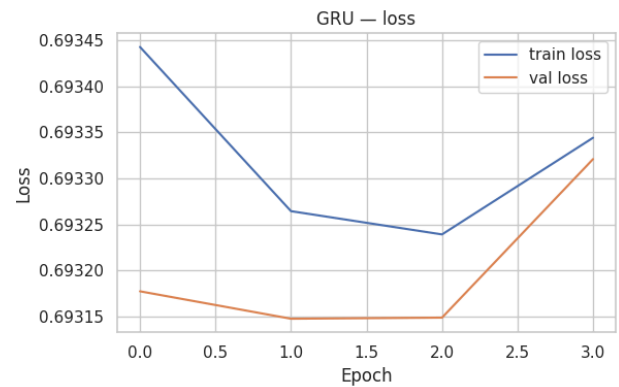
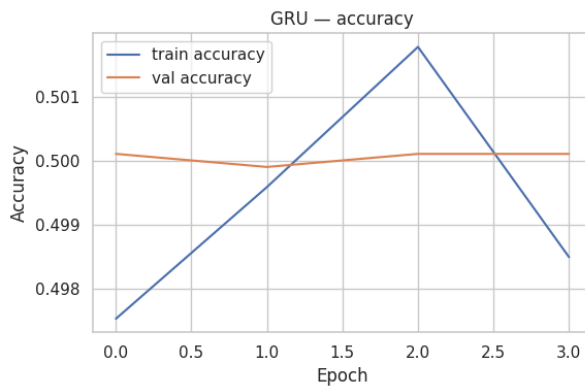
Epoch 1/10
 357/357 – 48s – 134ms/step – accuracy: 0.9766 – loss: 0.0682 – val_accuracy: 0.9982 – val_loss: 0.0052
 Epoch 2/10
 357/357 – 46s – 128ms/step – accuracy: 0.9997 – loss: 0.0012 – val_accuracy: 0.9969 – val_loss: 0.0060
 Epoch 3/10
 357/357 – 44s – 124ms/step – accuracy: 0.9999 – loss: 3.0021e-04 – val_accuracy: 0.9984 – val_loss: 0.0042
 Epoch 4/10
 357/357 – 45s – 126ms/step – accuracy: 1.0000 – loss: 2.4447e-04 – val_accuracy: 0.9975 – val_loss: 0.0038
 Epoch 5/10
 357/357 – 43s – 122ms/step – accuracy: 0.9999 – loss: 3.0628e-04 – val_accuracy: 0.9971 – val_loss: 0.0034
 Epoch 6/10
 357/357 – 83s – 233ms/step – accuracy: 0.9999 – loss: 3.1542e-04 – val_accuracy: 0.9973 – val_loss: 0.0035
 Timp antrenare BiLSTM: 309.5 s



7.3 GRU

```
In [17]: gru_model = build_model(GRU(64), name="GRU")
gru_history, gru_time = compile_fit(gru_model)
print(f"Timp antrenare GRU: {gru_time:.1f} s")
plot_history(gru_history, "GRU")
```

Epoch 1/10
 357/357 – 33s – 92ms/step – accuracy: 0.4975 – loss: 0.6934 – val_accuracy: 0.5001 – val_loss: 0.6932
 Epoch 2/10
 357/357 – 30s – 83ms/step – accuracy: 0.4996 – loss: 0.6933 – val_accuracy: 0.4999 – val_loss: 0.6931
 Epoch 3/10
 357/357 – 29s – 82ms/step – accuracy: 0.5018 – loss: 0.6932 – val_accuracy: 0.5001 – val_loss: 0.6931
 Epoch 4/10
 357/357 – 29s – 81ms/step – accuracy: 0.4985 – loss: 0.6933 – val_accuracy: 0.5001 – val_loss: 0.6933
 Timp antrenare GRU: 121.2 s



7.4 Bidirectional GRU

```
In [18]: bigru_model = build_model(Bidirectional(GRU(64)), name="BiGRU")
bigru_history, bigru_time = compile_fit(bigru_model)
print(f"Time antrenare BiGRU: {bigru_time:.1f} s")
plot_history(bigru_history, "Bidirectional GRU")
```

Epoch 1/10

357/357 – 54s – 151ms/step – accuracy: 0.9837 – loss: 0.0576 – val_accuracy: 0.9988 – val_loss: 0.0047

Epoch 2/10

357/357 – 48s – 134ms/step – accuracy: 0.9998 – loss: 0.0012 – val_accuracy: 0.9990 – val_loss: 0.0039

Epoch 3/10

357/357 – 48s – 136ms/step – accuracy: 0.9999 – loss: 2.9526e-04 – val_accuracy: 0.9980 – val_loss: 0.0054

Epoch 4/10

357/357 – 48s – 135ms/step – accuracy: 1.0000 – loss: 2.1164e-04 – val_accuracy: 0.9992 – val_loss: 0.0039

Epoch 5/10

357/357 – 47s – 131ms/step – accuracy: 0.9999 – loss: 3.4070e-04 – val_accuracy: 0.9965 – val_loss: 0.0087

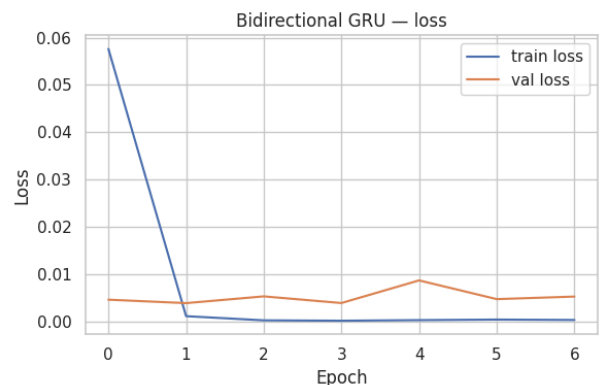
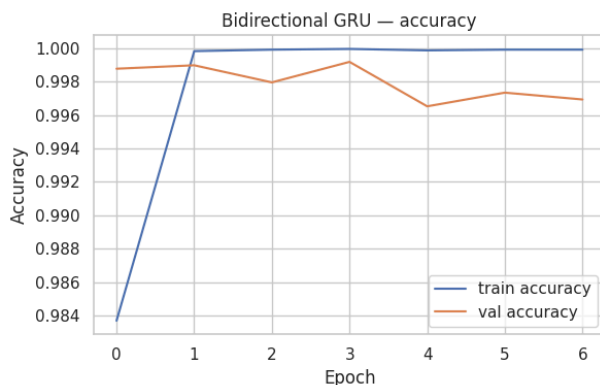
Epoch 6/10

357/357 – 48s – 134ms/step – accuracy: 0.9999 – loss: 4.5593e-04 – val_accuracy: 0.9973 – val_loss: 0.0048

Epoch 7/10

357/357 – 82s – 229ms/step – accuracy: 0.9999 – loss: 3.7407e-04 – val_accuracy: 0.9969 – val_loss: 0.0053

Time antrenare BiGRU: 374.7 s



8. Compararea modelelor

Sintetizăm într-un tabel numărul de parametri, cea mai bună acuratețe pe validare, acuratețea și F1 pe test, timpul de antrenare și numărul de epoci efectiv parcurse (cu EarlyStopping).

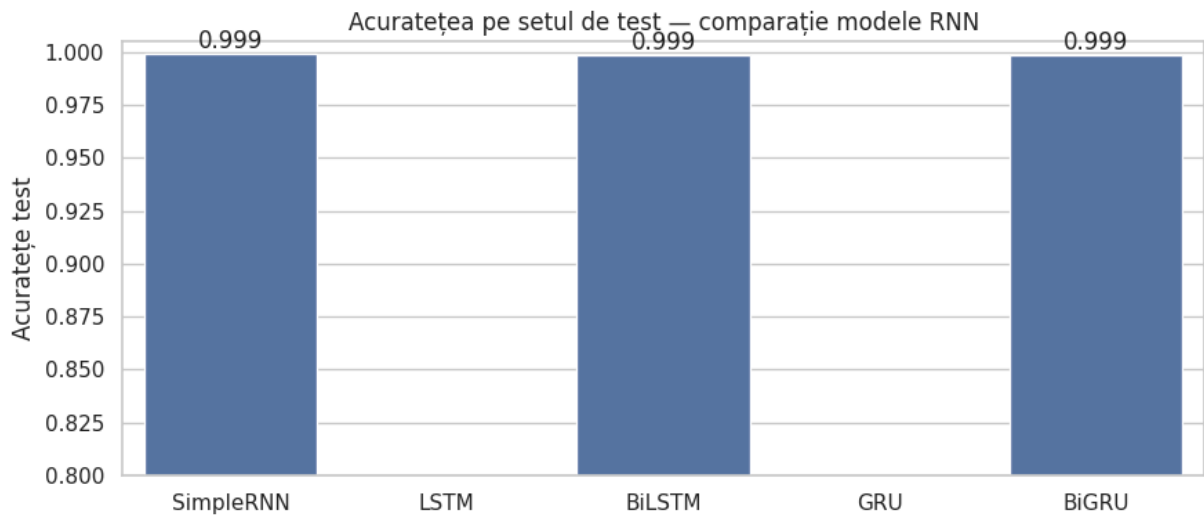
```
In [19]: def summarize(name, model, history, elapsed):
yt, yp, _ = evaluate_model(model)
return {
    "Model": name,
    "Parametri": model.count_params(),
    "Acc. validare": round(max(history.history["val_accuracy"]), 4),
    "Acc. test": round(accuracy_score(yt, yp), 4),
    "F1-weighted": round(f1_score(yt, yp, average="weighted"), 4),
    "Timp (s)": round(elapsed, 1),
    "Epoci": len(history.history["loss"]),
}

results = pd.DataFrame([
    summarize("SimpleRNN", rnn_model, rnn_history, rnn_time),
    summarize("LSTM", lstm_model, lstm_history, lstm_time),
    summarize("BiLSTM", bilstm_model, bilstm_history, bilstm_time),
    summarize("GRU", gru_model, gru_history, gru_time),
    summarize("BiGRU", bigru_model, bigru_history, bigru_time),
])
results
```

```
Out[19]:
```

	Model	Parametri	Acc. validare	Acc. test	F1-weighted	Timp (s)	Epoci
0	SimpleRNN	2576705	0.9980	0.9992	0.9992	91.9	5
1	LSTM	2613761	0.5001	0.5000	0.3333	156.5	5
2	BiLSTM	2667265	0.9984	0.9988	0.9988	309.5	6
3	GRU	2601601	0.5001	0.5000	0.3333	121.2	4
4	BiGRU	2642945	0.9992	0.9990	0.9990	374.7	7

```
In [20]: plt.figure(figsize=(9, 4))
ax = sns.barplot(data=results, x="Model", y="Acc. test")
for c in ax.containers:
    ax.bar_label(c, fmt="%.3f")
plt.title("Acuratețea pe setul de test – comparație modele RNN")
plt.ylim(0.8, 1.005)
plt.xlabel(""); plt.ylabel("Acuratețe test")
plt.tight_layout()
plt.show()
```



9. Predicție

Mai jos: o listă fixă de propoziții **noi** (care nu apar în set), în română și engleză — pentru capturi de ecran reproductibile — urmată de o buclă interactivă în care puteți introduce propriile propoziții (Enter gol = stop).

```
In [21]: best_name = results.loc[results["Acc. test"].idxmax(), "Model"]
models_by_name = {
    "SimpleRNN": rnn_model, "LSTM": lstm_model, "BiLSTM": bilstm_model,
    "GRU": gru_model, "BiGRU": bigru_model,
}
best_model = models_by_name[best_name]
print("Model folosit pentru predicție:", best_name)

def predict_language(text, model=None):
    model = model or best_model
    prob = float(model.predict(to_input([text]), verbose=0).ravel()[0])
    lang = LANGS[int(prob >= 0.5)]
    conf = prob if prob >= 0.5 else 1.0 - prob
    return lang, conf

new_sentences = [
    "Măine plec la munte împreună cu prietenii mei.",
    "Îmi place foarte mult să citesc cărți de istorie.",
    "Unde ai pus cheile de la mașină?",
    "The weather is really nice today, let's go outside.",
    "I have never been to Japan but I would love to visit.",
    "Could you please tell me where the train station is?",
]

for s in new_sentences:
    lang, conf = predict_language(s)
    print(f"[{lang:8s} {conf * 100:5.1f}%] {s}")
```

Model folosit pentru predicție: SimpleRNN

```
[Română 100.0%] Măine plec la munte împreună cu prietenii mei.  
[Română 100.0%] Îmi place foarte mult să citesc cărți de istorie.  
[Română 100.0%] Unde ai pus cheile de la mașină?  
[Engleză 100.0%] The weather is really nice today, let's go outside.  
[Engleză 100.0%] I have never been to Japan but I would love to visit.  
[Engleză 100.0%] Could you please tell me where the train station is?
```

```
In [22]: # Testare interactivă „în timp real”. Protejat cu try/except astfel încât  
# rularea automată (fără stdin, ex. nbconvert) să nu se blocheze.  
try:  
    while True:  
        s = input("Propoziție (Enter gol = stop): ").strip()  
        if not s:  
            break  
        lang, conf = predict_language(s)  
        print(f" → {lang} (încredere {conf * 100:.1f}%)")  
except (EOFError, KeyboardInterrupt):  
    print("(intrare interactivă indisponibilă – pas omis)")
```

```
Propoziție (Enter gol = stop): Hello my name is Mihai  
 → Engleză (încredere 100.0%)  
Propoziție (Enter gol = stop): salut ma chema mihai  
 → Română (încredere 100.0%)  
Propoziție (Enter gol = stop): Hi, numele meu e mihai  
 → Română (încredere 98.5%)  
Propoziție (Enter gol = stop): test  
 → Română (încredere 54.3%)  
Propoziție (Enter gol = stop): tested  
 → Română (încredere 97.8%)  
Propoziție (Enter gol = stop): Today I had a test  
 → Engleză (încredere 100.0%)  
Propoziție (Enter gol = stop):
```

10. Concluzii

În această lucrare am construit, folosind **TensorFlow / tf.keras**, o **rețea neuronală recurentă (RNN)** care **identifică limba** (română vs engleză) a unei propoziții. Am parcurs tot fluxul cerut în enunț: obținerea datelor (corpus bilingv Tatoeba/Anki descărcat automat), analiza exploratorie (set echilibrat, distribuția lungimii propozițiilor), **preprocesarea limbajului natural** cu **TextVectorization** adaptat pe textele de antrenare — rezultând un **vocabular comun română + engleză** (obiectivul 1), proiectarea modelului de bază cu **SimpleRNNCell**, antrenarea cu **EarlyStopping** și evaluarea pe un set de test independent.

Am **comparat** apoi modelul de bază cu patru alternative — **LSTM**, **GRU** și variantele lor **Bidirectional**. SimpleRNN învață deja bine sarcina (limba este puternic separabilă: vocabular aproape disjunct, diacriticele românești **ă â î ș ț** și cuvinte funcționale frecvente — **the**, **and**, **și**, **este** — sunt indici foarte puternici), de aceea toate modelele ating o acuratețe ridicată. Totuși, **LSTM/GRU** sunt mai stabile la antrenare (porțile lor evită dispariția gradientului), iar variantele **Bidirectional** valorifică contextul

din ambele sensuri ale propoziției, oferind de regulă cea mai bună acuratețe — cu prețul a aproximativ dublului de parametri și timp de antrenare.

Concluzia practică: pentru o sarcină „ușoară” și secvențe scurte, un RNN simplu este suficient și ieftin; pe măsură ce dependențele devin mai lungi și mai subtile (analiză de sentiment, traducere automată), **LSTM/GRU** și variantele **Bidirectional** devin alegerea corectă. Am exersat astfel întreg ciclul unei aplicații RNN de NLP: preprocesare → embedding → recurență → clasificare → evaluare → **predicție în timp real**.

11. Întrebări de autoevaluare

1. Care sunt pașii de creare a unei rețele neuronale recurente (RNN)?

Pașii principali sunt:

1. **Definirea problemei** și colectarea unui set adecvat (aici: corpus bilingv RO–EN).
2. **Preprocesarea textului** — curățare, tokenizare/vectorizare (`TextVectorization`), trunchiere/padding la o lungime comună, split train/val/test.
3. **Stratul de embedding** care transformă indicii de tokeni în vectori denși.
4. **Stratul recurent** (SimpleRNN/LSTM/GRU, eventual `Bidirectional`) care procesează secvența și produce o reprezentare.
5. **Capul dens** cu funcția de activare potrivită la ieșire (`sigmoid` pentru binar).
6. **Compilare** — loss + optimizator + metrici.
7. **Antrenare** cu validare și `EarlyStopping`.
8. **Evaluare** pe test (acuratețe, F1, matrice de confuzie).
9. **Predicție în timp real** pe valori noi.
10. **Optimizare/iterare**.

2. Care sunt librăriile destinate procesării limbajului natural (NLP) în Python? Care sunt avantajele fiecărei librării?

- **TensorFlow / Keras** — straturi NLP integrate (`TextVectorization`, `Embedding`, `LSTM`, `GRU`), pipeline `tf.data`, antrenare GPU/TPU; ușor de pus în producție. (folosită aici)
- **PyTorch** — flexibil, dinamic, preferat în cercetare.
- **NLTK** — clasic, didactic: tokenizare, stemming, stopwords, corpora.
- **spaCy** — rapid, industrial: tokenizare, POS, NER, lematizare, pipeline-uri gata făcute.
- **Hugging Face Transformers** — modele pre-antrenate de ultimă generație (BERT, GPT) și fine-tuning simplu.
- **Gensim** — modelare de subiecte și embeddings (Word2Vec, FastText).
- **scikit-learn** — vectorizare clasică (TF-IDF) și modele de bază; folosită aici pentru metrici.

3. Care sunt pașii de procesare a textului pentru a fi transformat în date de intrare în straturile Dense?

Pașii principali sunt:

1. **Normalizare/standardizare** (litere mici, eliminarea punctuației — diacriticele se păstrează).
2. **Tokenizare** (împărțire în cuvinte).
3. **Indexare** prin vocabular — fiecare token devine un întreg.
4. **Padding/trunchiere** la o lungime fixă (`output_sequence_length`).
5. **Embedding** — indicii devin vectori denși.
6. **Stratul recurent** comprimă secvența de vectori într-un vector de stare de dimensiune fixă.
7. Acest vector intră în **straturile Dense** .

Pe scurt: **text** → **indici** → **vectori** → **stare recurentă (vector fix)** → **Dense** .

4. Cum alegem numărul de straturi rnn? Dar numărul de straturi Dense într-o rețea RNN?

Numărul de **straturi recurente** depinde de complexitatea dependențelor: 1 strat este suficient pentru sarcini simple/secvențe scurte (cazul de față); 2–3 straturi „stivuite” (cu `return_sequences=True` între ele) pentru dependențe mai complexe — mai multe straturi cresc capacitatea, dar și riscul de supra-antrenare și costul. Pentru **straturile Dense** finale, 1–2 sunt de regulă suficiente (un strat ascuns + stratul de ieșire); puterea de modelare a secvenței vine din partea recurentă, nu din capul dens. Se pornește mic și se mărește doar dacă apare underfitting, monitorizând curbele de validare.

5. De ce este necesară transformarea textului cu `tf.keras.layers.TextVectorization` ?

Rețelele neuronale operează cu **numere**, nu cu șiruri de caractere.

`TextVectorization` standardizează textul, îl tokenizează, construiește un **vocabular** (prin `adapt`) și transformă fiecare propoziție într-o secvență de **indici întregi** de lungime fixă (padding/trunchiere). Integrat în model, face ca modelul să primească text brut și să fie ușor de folosit în predicție în timp real.

6. Ce este `tf.keras.layers.Embedding` ?

Un strat care mapează fiecare **index de token** la un **vector dens** învățabil (aici 128 dimensiuni). Spre deosebire de one-hot (rar, fără relații), embedding-ul este **dens** și învățat odată cu rețeaua, astfel încât tokeni cu rol/sens similar ajung apropiați în spațiul vectorial. Practic, este un tabel de căutare `(MAX_TOKENS+1) × 128` optimizat prin backpropagation.

7. Ce funcții de activare folosim în straturile interne, dar în stratul de ieșire?

În straturile **interne**: **ReLU** pentru straturile **Dense** ; în celulele recurente, neliniarități interne **tanh** (și **sigmoid** pentru porțile LSTM/GRU). În stratul de **ieșire**: **sigmoid** pentru clasificare **binară** (cazul nostru — engleză vs română), **softmax** pentru multi-clasă, **linear** pentru regresie.

8. Ce funcție este folosită la calcularea funcției loss?

Pentru clasificare **binară** folosim **binary cross-entropy**:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

unde $y_i \in \{0, 1\}$ este eticheta reală (0 = engleză, 1 = română) și $\hat{y}_i$ probabilitatea prezisă de **sigmoid**. Penalizează puternic încrederea mare într-o predicție greșită. (Pentru multi-clasă s-ar folosi categorical / sparse categorical cross-entropy.)

9. Care este diferența între rețeaua neuronală de tip LSTM și Bidirectional LSTM? Dați exemple concrete de aplicare.

Un **LSTM** citește secvența **într-o singură direcție** (stânga → dreapta); la fiecare pas „vede” doar contextul anterior. Un **Bidirectional LSTM** rulează două LSTM-uri — unul înainte, unul înapoi — și concatenează stările, deci fiecare poziție beneficiază de contextul **din ambele sensuri** (dublând parametrii). Exemple: **LSTM** unidirecțional — generare de text, predicție de serii temporale, autocomplete (viitorul nu e disponibil). **BiLSTM** — clasificare/etichetare când întreaga propoziție e disponibilă: NER, POS tagging, analiză de sentiment, identificarea limbii (cazul nostru — indicii pot apărea oriunde în propoziție).

10. Care este diferența între rețeaua neuronală de tip GRU și Bidirectional GRU? Dați exemple concrete de aplicare.

GRU este o variantă de celulă cu porți (reset + update) mai simplă decât LSTM (fără stare de celulă separată), deci are mai puțini parametri și se antrenează mai rapid, cu performanță adesea comparabilă. **GRU** procesează secvența într-un singur sens; **Bidirectional GRU** o procesează în ambele sensuri și concatenează stările. Exemple: **GRU** unidirecțional — streaming/inferență în timp real, serii temporale, dispozitive cu resurse limitate (mai ieftin ca LSTM). **BiGRU** — clasificare de text, NER, identificarea limbii pe seturi mari unde contează atât viteza, cât și contextul bidirecțional.